
Manual de Tecnico KQMIP

Cálculo Heurístico de Información Integrada (φ)

KQNodes / KGeoMIP / GeoMIP Genético

Autor:

Isabel Sofia Benitez Duque
isabel.benitez@ucaldas.edu.co

Docente:

Luz Enith Guerrero Mendieta
Departamento de Sistemas

Índice

1. Introducción y Alcance	3
1.1. Nomenclatura	3
2. Fundamentos Matemáticos de IIT	4
2.1. Sistema Binario y Matriz de Transición	4
2.2. k -Particiones	4
2.3. Distancia de Mover la Tierra (EMD)	5
2.4. La Medida φ y el Problema k -MIP	6
3. Arquitectura del Sistema	7
3.1. Capas del Sistema	7
3.2. Flujo de Ejecución	8
3.3. Aislamiento Subprocess	8
4. Estructura del Repositorio	9
5. Módulo de Interfaz Gráfica (gui/)	10
5.1. <code>app.py</code> — Ventana Principal	10
5.2. <code>runner.py</code> — Lógica de Ejecución	10
5.3. <code>validator.py</code>	11
6. Módulo de Workers (gui/logica/)	12
6.1. Protocolo de Comunicación	12
6.2. Workers disponibles	12
7. Algoritmo KQNodes	13
7.1. Funciones Submodulares	13
7.2. Teorema de Queyranne	13
7.3. Función Submodular en KQNodes	14
7.4. Métrica Utilizada por KQNodes	14
7.5. Descripción y Pseudocódigo	15
7.6. Complejidad	15
7.7. Ubicación en el Código	15
8. Algoritmo KGeoMIP	16
8.1. El Hipercubo de Hamming	16
8.2. Tabla de Costos de Transición	16
8.3. Bisección Greedy Recursiva	17
8.4. Ubicación en el Código	18
9. Algoritmo GeneticKGeoMIP	19
9.1. Motivación	19
9.2. Espacio de Búsqueda y Codificación	19
9.3. Función de Aptitud	19
9.4. Operadores Genéticos	20
9.5. Parámetros Adaptativos	20
9.6. Complejidad y Comparación con KGeoMIP	20

9.7. Ubicación en el Código	21
10. Clase Base SIA y Núcleo Compartido	22
10.1. Clase SIA	22
10.2. Clase Solution	22
10.3. Funciones Clave del Núcleo	22
11. Módulo de Ejecución por Lotes (scripts/run_batch.py)	23
11.1. Descripción	23
11.2. Diferencias con la GUI	23
11.3. Uso	23
12. API de Referencia	23
12.1. Interfaz Común	23
12.2. Ejemplo de Uso Directo	23
12.3. Parámetros Adicionales de GeneticKGeoMIP	24
13. Análisis de Complejidad	25
13.1. Guía de Selección de Algoritmo	26
14. Limitaciones y Trabajo Futuro	27
14.1. Limitaciones Conocidas	27
14.2. Trabajo Futuro	27

1. Introducción y Alcance

Este manual documenta la arquitectura interna, los fundamentos matemáticos y la API del sistema KQMIP. Está dirigido a desarrolladores o estudiantes con conocimientos básicos de Python y álgebra lineal que necesiten entender, extender o integrar las estrategias de partición implementadas.

Idea central. Dado un sistema de n nodos interconectados (por ejemplo, neuronas o compuertas lógicas), queremos saber *hasta qué punto el sistema funciona como un todo* o si puede describirse como la unión de partes independientes. Para medirlo se usa φ (phi): si al partir el sistema en k grupos la información que se pierde es muy poca, el sistema es casi separable; si φ es grande, las partes están fuertemente integradas. Encontrar la partición que *menos* φ produce — la k -MIP — es el problema central de KQMIP.

El proyecto extiende dos implementaciones originales de bipartición (GeoMIP y QNodes) al caso general de k grupos ($k \geq 2$).

1.1. Nomenclatura

Nombre	Significado	Ubicación
KQNodes	Queyranne extendido a k -particiones	QNodes/src/strategies/q_nodes.py
KGeoMIP	Greedy recursivo para k -particiones	GeoMIP/.../strategies/geometric.py
GeneticKGeoMIP	Genético heurístico para k -particiones	GeoMIP/.../strategies/genetic_optimizer.py

2. Fundamentos Matemáticos de IIT

Esta sección presenta las definiciones formales que usan los tres algoritmos. Si ya conoce IIT puede saltar directamente a la Sección ??.

2.1. Sistema Binario y Matriz de Transición

En palabras sencillas. El sistema se compone de n nodos, cada uno con estado 0 (apagado) o 1 (encendido). La TPM es una tabla con 2^n filas (una por cada combinación posible de estados) y n columnas (una por nodo). La celda (v, i) dice: “si el sistema está en el estado v ahora, ¿con qué probabilidad estará el nodo i encendido en el siguiente instante?”

Definición 2.1 (Sistema Binario). Un **sistema binario** de n nodos es una tupla (V, M) donde:

- $V = \{1, \dots, n\}$ es el conjunto de nodos.
- $M \in [0, 1]^{2^n \times n}$ es la **Matriz de Probabilidad de Transición** (TPM), con $M_{v,i} = P(X_{i,t+1} = 1 \mid X_t = v)$ para cada $v \in \{0, 1\}^n$ y $i \in V$.

El espacio de estados es $\Omega = \{0, 1\}^n$ con $|\Omega| = 2^n$.

Cuando los nodos son condicionalmente independientes entre sí (dado el estado actual), la probabilidad de que el sistema pase al estado w desde v_t es simplemente el producto de las probabilidades individuales de cada nodo:

$$p(w \mid v_t) = \prod_{i=1}^n M_{v_t,i}^{w_i} (1 - M_{v_t,i})^{1-w_i}, \quad w \in \{0, 1\}^n \quad (1)$$

Definición 2.2 (Subsistema mecanismo-purview). En la práctica no siempre se analiza el sistema completo. El **mecanismo** $C \subseteq V$ son los nodos que observamos ahora (t), y el **purview** o **alcance** $A \subseteq V$ son los nodos cuyo futuro ($t+1$) queremos predecir. La distribución del purview dado el estado fijo v_t es:

$$p_A(a \mid v_t) = \prod_{i \in A} M_{v_t,i}^{a_i} (1 - M_{v_t,i})^{1-a_i}, \quad a \in \{0, 1\}^{|A|}$$

2.2. k -Particiones

En palabras sencillas. Partir el sistema en k grupos significa asignar cada nodo a exactamente uno de k subconjuntos, sin que haya nodos sin grupo ni nodos repetidos. Por ejemplo, con 4 nodos $\{A, B, C, D\}$ y $k = 2$: una partición válida es $\{\{A, B\}, \{C, D\}\}$; otra es $\{\{A\}, \{B, C, D\}\}$.

Definición 2.3 (k -Partición). Una **k -partición** de V es $\pi = \{S_1, \dots, S_k\}$ con:

$$S_i \neq \emptyset, \quad \bigcup_{i=1}^k S_i = V, \quad S_i \cap S_j = \emptyset \quad \forall i \neq j$$

Denotamos $\Pi_k(V)$ al conjunto de todas las k -particiones de V .

El número total de k -particiones posibles crece muy rápido con n y k , lo que hace inviable revisarlas todas (búsqueda exhaustiva). La fórmula exacta usa los números de Stirling de segundo tipo:

Proposición 2.1 (Cardinalidad de $\Pi_k(V)$).

$$|\Pi_k(V)| = S(n, k) = \frac{1}{k!} \sum_{j=0}^k (-1)^{k-j} \binom{k}{j} j^n$$

Ejemplo 2.1. Para $n = 10$: $S(10, 2) = 511$; $S(10, 3) = 9\,330$; $S(15, 3) \approx 2,4 \times 10^6$; $S(20, 3) \approx 5,8 \times 10^8$. La búsqueda exhaustiva es inviable para $k \geq 3$, $n \gtrsim 15$.

Definición 2.4 (Distribución Particionada). Al partir el sistema en k grupos, se *simula* que esos grupos no interactúan entre sí: la probabilidad del sistema completo se calcula multiplicando las probabilidades de cada grupo por separado.

$$p_\pi(w \mid v_t) = \prod_{i=1}^k p_{S_i}(w_{S_i} \mid v_t), \quad w \in \{0, 1\}^n \quad (2)$$

Si el sistema *realmente* fuera separable en esos grupos, p_π coincidiría con p . La diferencia entre ambas es lo que mide φ .

2.3. Distancia de Mover la Tierra (EMD)

En palabras sencillas. El EMD mide cuán distintas son dos distribuciones de probabilidad. La idea intuitiva: imagine que p y q son dos maneras de distribuir arena sobre el piso. El EMD es el costo mínimo de mover la arena de la forma p a la forma q , donde el costo de mover un gránulo depende de la distancia que recorre.

La **distancia de Hamming** $d_H(x, y)$ entre dos estados binarios es simplemente cuántos bits difieren: estado 0011 vs 1010 difiere en 3 bits, entonces $d_H = 3$.

Definición 2.5 (Métrica de Hamming).

$$d_H(x, y) = \sum_{i=1}^n |x_i - y_i|, \quad x, y \in \{0, 1\}^n$$

Definición 2.6 (Earth Mover's Distance). Sean p, q distribuciones sobre $\{0, 1\}^n$. El EMD es el costo mínimo de transportar la probabilidad de p hasta quedar distribuida como q , donde transportar probabilidad entre dos estados cuesta su distancia de Hamming:

$$\text{EMD}(p, q) = \min_{\gamma \in \Gamma(p, q)} \sum_{x, y} d_H(x, y) \gamma(x, y) \quad (3)$$

donde $\gamma(x, y)$ es el plan de transporte: cuánta probabilidad se mueve desde el estado x al estado y . El conjunto $\Gamma(p, q)$ contiene todos los planes válidos (los que respetan los totales de p y q):

$$\Gamma(p, q) = \left\{ \gamma \geq 0 : \sum_y \gamma(x, y) = p(x), \sum_x \gamma(x, y) = q(y) \right\}$$

El cálculo de este mínimo es un problema de programación lineal. En la práctica se resuelve con la librería `pyemd`.

Proposición 2.2 (Factorización del EMD bajo independencia). *Si los nodos del purview son condicionalmente independientes dado v_t y la partición respeta fronteras de nodos (node-partition), el EMD admite evaluación marginal:*

$$\text{EMD}(p(\cdot \mid v_t), p_\pi(\cdot \mid v_t)) = \sum_{i \in A} |p(X_i=1 \mid v_t) - p_\pi(X_i=1 \mid v_t)|$$

Nota 2.1 (Diferencia crítica entre implementaciones). KQNodes y KGeoMIP utilizan métricas distintas e incomparables:

Aspecto	KQNodes	KGeoMIP / Genético
Métrica φ	L^1 sobre marginales por nodo	EMD Wasserstein-1 con métrica Hamming
Fórmula	$\sum_{i \in A} p(X_i=1 \mid v_t) - p_\pi(X_i=1 \mid v_t) $	$\text{EMD}(p(\cdot \mid v_t), p_\pi(\cdot \mid v_t))$
Rango	$[0, A]$ donde $ A $ = tamaño del alcance	$[0, n]$ aprox.; depende de las distribuciones
Espacio de particiones	Cross-partitions: cada nodo puede estar en distintas partes en t y $t+1$	Node-partitions: cada nodo permanece en la misma parte en t y $t+1$
Tamaño del espacio	$2^{u+v-1} - 1$ (u nodos en t , v en $t+1$)	$2^{n-1} - 1$ biparticiones de nodos
Implementación	<code>emd_efecto(u,v) = np.sum(np.abs(u-v))</code>	<code>pyemd.emd(u, v, hamming_costs)</code>

Los valores absolutos de φ **no son comparables** entre KQNodes y KGeoMIP. Para normalizar KQNodes a $[0, 1]$: $\varphi_{\text{norm}} = \varphi/|A|$.

2.4. La Medida φ y el Problema k -MIP

En palabras sencillas. φ (phi) mide la información que se pierde al tratar el sistema como si sus k partes fueran independientes. Si $\varphi \approx 0$, la partición elegida es “casi correcta”: el sistema realmente funciona como si las partes no se comunicaran. Si φ es grande, la partición es mala: las partes sí interaccionan significativamente.

El objetivo es encontrar la partición que produce el φ más pequeño posible, llamada k -MIP (Minimum Information Partition).

Definición 2.7 (Información Integrada φ). φ es la distancia entre la distribución real del sistema p y la distribución que tendría si las k partes fueran independientes p_π :

$$\varphi(\pi, v_t) = \text{EMD}[p(\cdot \mid v_t), p_\pi(\cdot \mid v_t)] \geq 0 \quad (4)$$

Definición 2.8 (k -MIP). La k -MIP es la partición que minimiza φ entre todas las posibles k -particiones del sistema:

$$\pi_k^* = \arg \min_{\pi \in \Pi_k(V)} \varphi(\pi, v_t), \quad \varphi_k^* = \varphi(\pi_k^*, v_t) \quad (5)$$

Nota 2.2 (Dificultad computacional). Encontrar la k -MIP exacta requiere evaluar *todas* las posibles k -particiones. Según la Proposición 2.1, ese número crece tan rápido que para $k \geq 3$ y $n \gtrsim 15$ ya no es viable hacerlo en tiempo razonable (problema NP-difícil). Por eso los tres algoritmos de KQMIP producen soluciones *aproximadas* $\hat{\pi}_k$ con $\varphi(\hat{\pi}_k) \geq \varphi_k^*$.

3. Arquitectura del Sistema

3.1. Capas del Sistema

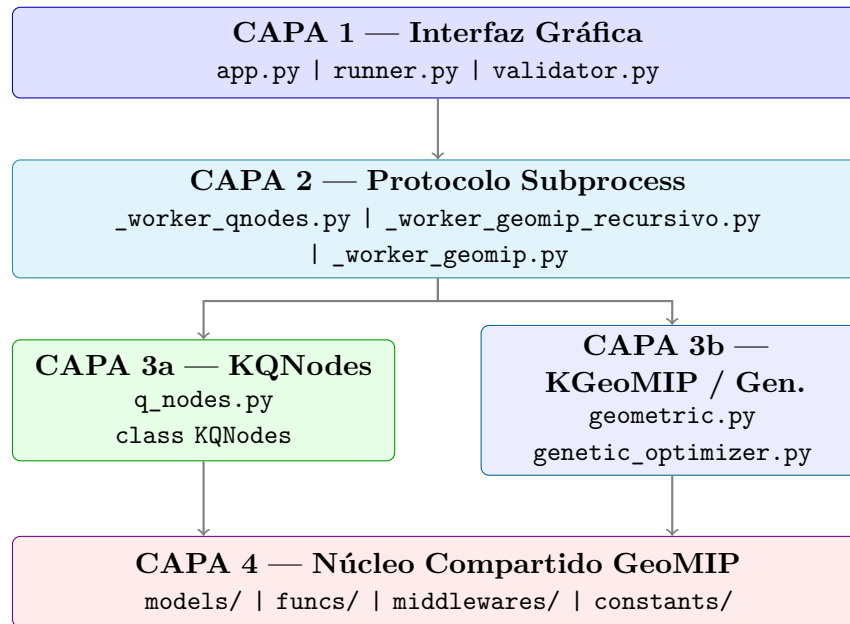


Figura 1: Capas de la arquitectura KQMIP. Las capas 3a y 3b son motores independientes que comparten el núcleo de la capa 4.

3.2. Flujo de Ejecución

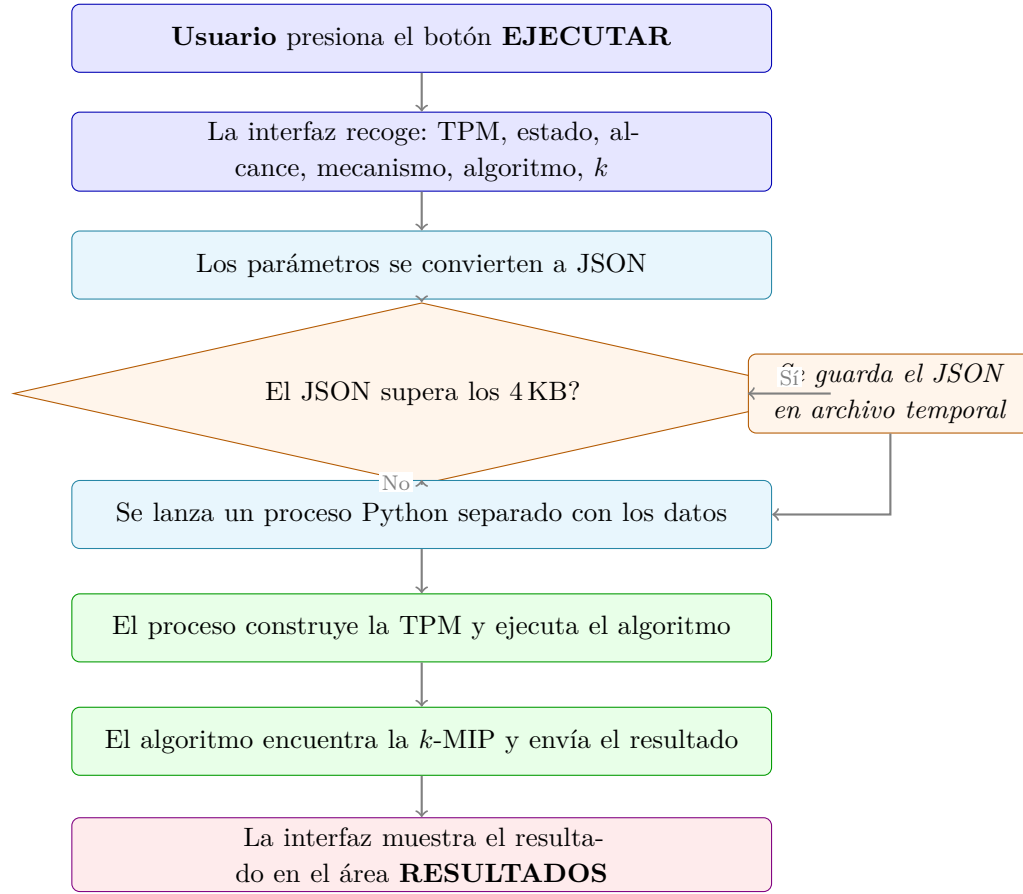


Figura 2: Flujo de ejecución de KQMIP. Cuando la TPM es grande ($n \geq 10$), los datos se guardan en un archivo temporal en lugar de pasarse por la línea de comandos.

3.3. Aislamiento Subprocess

Cada cálculo corre en un proceso Python separado (no en la misma memoria que la GUI). Esto tiene tres ventajas prácticas:

- **Estabilidad:** si el cálculo falla o consume demasiada memoria, la GUI no se cierra ni se congela.
- **Cancelación limpia:** `Runner.cancel()` envía una señal de terminación (`SIGTERM`) al proceso, que se detiene de inmediato.
- **Sin bloqueo de la interfaz:** Python tiene un bloqueo global (GIL) que impide que los cálculos pesados corran en hilos normales. Al usar un proceso separado, el CPU puede trabajar a plena capacidad sin congelar los botones o el área de texto.

Para sistemas con $n \geq 10$, la TPM tiene $2^n \times n$ números en punto flotante y su representación JSON supera los 4096 bytes (límite de línea de comandos en Windows). En ese caso, el JSON se escribe en un archivo temporal y se pasa como `@{ruta}`, evitando el error `WinError 206`.

4. Estructura del Repositorio

Listing 1: Árbol de directorios del proyecto

```
projecto-analisis-20261/
|-- main.py                                <- python main.py
|-- requirements.txt
|
|-- gui/
|   |-- app.py                            <- Ventana principal (
|       customtkinter)
|   |-- runner.py                         <- Subprocess + carga de datos
|   |-- validator.py                     <- Validacion de parametros
|   '-- logica/
|       |-- _worker_qnodes.py            <- Worker KQNodes
|       |-- _worker_geomip_recurativo.py <- Worker KGeoMIP (greedy)
|       '-- _worker_geomip.py            <- Worker GeneticKGeoMIP
|
|-- QNodes/
|   '-- src/strategies/q_nodes.py        <- class KQNodes
|
|-- GeoMIP/src/kgeomip/
|   '-- src/
|       |-- controllers/
|       |   |-- manager.py               <- class Manager
|       |   '-- strategies/
|       |       |-- geometric.py         <- class KGeoMIP
|       |       '-- genetic_optimizer.py <- class GeneticKGeoMIP
|       |-- models/base/sia.py           <- clase base SIA
|       '-- models/core/solution.py      <- class Solution
|
|-- scripts/run_batch.py                  <- Ejecucion por lotes
'-- docs/
```

5. Módulo de Interfaz Gráfica (gui/)

5.1. app.py — Ventana Principal

Cuadro 1: Métodos principales de App en app.py

Método	Responsabilidad
<code>__init__()</code>	Variables de estado, instancia Runner, construye UI
<code>_render_manual()</code>	Panel Manual: área TPM + campos binarios
<code>_render_csv()</code>	Panel CSV: selector de archivo
<code>_render_excel()</code>	Panel Excel: hoja, rango, n , semilla
<code>_collect_manual()</code>	Lee y valida parámetros del modo Manual
<code>_collect_csv()</code>	Carga TPM desde CSV y construye <code>ExecutionParams</code>
<code>_start_excel()</code>	Carga filas del Excel y prepara la primera <code>ExecutionParams</code>
<code>_on_run()</code>	Valida $\rightarrow$ llama <code>Runner.execute()</code> $\rightarrow$ actualiza estado UI
<code>_on_next()</code>	Avanza al siguiente ítem Excel y ejecuta
<code>_cb_result()</code>	Recibe <code>RunResult</code> , formatea y muestra resultados
<code>_on_save()</code>	Exporta resultados de la sesión a Excel

5.2. runner.py — Lógica de Ejecución

Atributo	Tipo	Descripción
<code>algo</code>	<code>str</code>	"qnodes" "geomip" "geomip-genetico"
<code>tpm</code>	<code>ndarray</code>	$2^n \times n$, <code>dtype=float</code>
<code>estado_inicial</code>	<code>str</code>	n bits
<code>condicion</code>	<code>str</code>	n bits
<code>alcance</code>	<code>str</code>	n bits
<code>mecanismo</code>	<code>str</code>	n bits
<code>ks</code>	<code>list[int]</code>	e.g. [2, 3, 4, 5]
<code>semilla</code>	<code>int</code>	Semilla para TPM aleatoria (default: 73)

`ExecutionParams.to_payload()` serializa todos los atributos a dict JSON, incluyendo `tpm_data`.

Callback	Cuándo se llama	Argumento
<code>on_tick(t)</code>	Cada 100 ms mientras el worker está activo	<code>float</code> — segundos
<code>on_result(r)</code>	Al recibir JSON válido por stdout	<code>RunResult</code>
<code>on_error(m)</code>	Si <code>returncode</code> $\neq 0$ o excepción	<code>str</code> — mensaje

Callbacks de Runner.

5.3. validator.py

Función	Qué valida
<code>validate_tpm(tpm)</code>	2D, $n \geq 2$, filas = 2^n , sin NaN
<code>validate_bit_fields(fields, n)</code>	Longitud n , solo "0" y "1"
<code>validate_ks(ks)</code>	Lista no vacía
<code>validate_n(n_str)</code>	$2 \leq n \leq 25$
<code>validate_seed(seed_str)</code>	Entero válido
<code>validate_excel_range(desde,hasta,tot)</code>	Rango dentro del total

6. Módulo de Workers (gui/logica/)

6.1. Protocolo de Comunicación

Cada worker sigue los mismos cuatro pasos al iniciarse:

1. **Leer los parámetros:** revisa el argumento de la línea de comandos (`argv[1]`). Si empieza con "@", lee el JSON desde un archivo temporal (para sistemas grandes); de lo contrario, lo parsea directamente.
2. **Configurar la ruta de Python:** agrega la carpeta raíz del motor (QNodes/ o GeoMIP/) al `sys.path` para poder importar las clases del algoritmo.
3. **Reconstruir la TPM:** si el JSON incluye los valores ("`tpm_data`"), los usa; si no, genera la TPM aleatoria con la semilla indicada (`np.random.seed(semilla)`).
4. **Ejecutar y devolver:** llama a `aplicar_estrategia()` e imprime el resultado como JSON en la salida estándar (`stdout`), que el proceso padre (la GUI) captura.

Estructura del JSON de salida:

Listing 2: Formato de salida de los workers

```
1 {  
2   "2": { "particion": "Parte 1: ...", "perdida": 0.0625, "tiempo":  
         1.21 },  
3   "3": { "particion": "Parte 1: ...\\nParte 2: ...", "perdida": 0.0312,  
         "tiempo": 0.94 }  
4 }
```

6.2. Workers disponibles

Listing 3: `_worker_qnodes.py`

```
1 QNODES_ROOT = Path(__file__).resolve().parents[2] / "QNodes"  
2 sys.path.insert(0, str(QNODES_ROOT))  
3 from src.strategies.q_nodes import KQNodes
```

Listing 4: `_worker_geomip_recurativo.py` (KGeoMIP greedy)

```
1 GEOMIP_ROOT = Path(__file__).resolve().parents[2] / "GeoMIP" / "src" /  
  "kgeomip"  
2 sys.path.insert(0, str(GEOMIP_ROOT))  
3 from src.controllers.strategies.geometric import KGeoMIP
```

Listing 5: `_worker_geomip.py` (GeneticKGeoMIP)

```
1 from src.controllers.strategies.genetic_optimizer import  
  GeneticKGeoMIP
```

7. Algoritmo KQNodes

Idea general. KQNodes fusiona nodos de a uno en cada paso, eligiendo siempre el que “menos información agrega” al grupo actual. Al terminar tiene los grupos que forman la mejor partición posible, y capta automáticamente los resultados para $k = 2, 3, 4, \dots$ en un solo recorrido. Esta estrategia es correcta porque la función φ tiene una propiedad especial llamada *submodularidad*, que garantiza que la fusión voraz funciona bien.

7.1. Funciones Submodulares

En palabras sencillas. Una función es submodular cuando “añadir un elemento a un grupo pequeño aporta más (o igual) que añadirlo a un grupo grande”. Es el principio de rendimiento marginal decreciente: el primer nodo que se añade a un grupo cambiará mucho el resultado; los siguientes, cada vez menos.

Definición 7.1 (Función Submodular). $f : 2^V \rightarrow \mathbb{R}$ es **submodular** si para todo $A, B \subseteq V$:

$$f(A) + f(B) \geq f(A \cup B) + f(A \cap B)$$

De forma equivalente, tiene **rendimientos marginales decrecientes**: agregar el mismo elemento x a un grupo más pequeño A produce un cambio mayor (o igual) que agregarlo a un grupo más grande $B \supseteq A$:

$$f(A \cup \{x\}) - f(A) \geq f(B \cup \{x\}) - f(B)$$

f es **simétrica** si $f(S) = f(V \setminus S)$ para todo S (el costo de un grupo es igual al costo de su complemento).

Proposición 7.1 (Submodularidad de φ en bipartición). *La función $f_{bp}(S) = \text{EMD}[p(\cdot \mid v_t), p_S \cdot p_{V \setminus S}]$ es simétrica y submodular bajo la métrica de Hamming. Esto es lo que permite aplicar el algoritmo de Queyranne eficientemente.*

7.2. Teorema de Queyranne

En palabras sencillas. Queyranne demostró que si la función que queremos minimizar es submodular y simétrica, se puede encontrar la bipartición mínima de forma eficiente (en tiempo polinomial), sin revisar las 2^{n-1} opciones. El algoritmo construye en cada fase una cadena de nodos y garantiza que el último de la cadena puede “separarse” del resto sin afectar la solución óptima.

Teorema 7.1 (Queyranne, 1998). *Para $f : 2^V \rightarrow \mathbb{R}$ submodular y simétrica con $n = |V|$ elementos, existe un algoritmo que encuentra la bipartición que minimiza f :*

$$S^* = \arg \min_{S \neq \emptyset, S \neq V} f(S)$$

usando $\mathcal{O}(n^3)$ evaluaciones de f y $\mathcal{O}(n^2)$ espacio en memoria.

La clave es el **par pendiente** (v_{n-1}, v_n) : en cada fase, el algoritmo ordena los nodos por cuánto contribuyen al grupo actual. El último nodo de ese orden siempre puede separarse o contraerse sin perder la solución óptima, lo que permite reducir el problema paso a paso.

7.3. Función Submodular en KQNodes

En cada paso, el algoritmo decide qué nodo u conviene fusionar con el grupo actual Ω . Para decidirlo, calcula cuánto mejora φ al agregar u : si Δ es el nodo candidato, la ganancia es:

$$g(\Delta, \Omega) = \varphi(\{\Delta\}, v_t) - \varphi(\{\Delta \cup \Omega\}, v_t) \quad (6)$$

El nodo con menor g es el que menos información agrega al grupo, y por eso se elige para fusionar.

Para evitar recalcular los mismos valores en fases posteriores, la implementación guarda en `memoria_delta` el par $(\text{emd}_\Delta, p_\Delta)$ de cada nodo ya evaluado.

7.4. Métrica Utilizada por KQNodes

Nota 7.1 (Implementación real de `emd_efecto`). A pesar del nombre, la función `emd_efecto` en `KQNodes/src/funcs/iit.py` **no** calcula el Earth Mover's Distance (Wasserstein-1). Su implementación es:

```
1 def emd_efecto(u, v):
2     return np.sum(np.abs(u - v))    # norma L1
```

donde u y v son vectores de $|A|$ entradas, uno por nodo del alcance (A), cada entrada siendo $P(\text{nodo}_i = \text{OFF} \mid v_t)$. Estos vectores provienen de `distribucion_marginal()`, que devuelve marginales **por nodo**, asumiendo independencia entre nodos.

Proposición 7.2 (Rango de φ en KQNodes). *Sea $|A|$ el número de nodos en el alcance. Para cualquier estado inicial v_t y cualquier partición:*

$$0 \leq \varphi_{\text{KQNodes}} \leq |A|$$

El máximo se alcanza cuando la distribución marginal de cada nodo cambia completamente al partir el sistema. Para comparaciones entre sistemas de distinto tamaño, se recomienda la forma normalizada:

$$\varphi_{\text{norm}} = \frac{\varphi_{\text{KQNodes}}}{|A|} \in [0, 1]$$

Nota 7.2 (Cross-partitions en KQNodes). KQNodes explora el espacio de **cross-partitions**: cada nodo puede pertenecer a distintas partes en t y en $t+1$ dentro de la misma partición. Este espacio tiene tamaño $2^{u+v-1} - 1$ (con u nodos en la causa y v nodos en el efecto), en contraste con las node-partitions de KGeoMIP (véase Nota 8.1). Esto hace que KQNodes explore un espacio de particiones más grande y fiel a la especificación IIT 3.0, pero con la métrica L^1 (aproximación al EMD exacto).

7.5. Descripción y Pseudocódigo

Algoritmo 1: KQNodes — variante secuencial de Queyranne para k -particiones

Input: Vértices V ;
valores $K = \{k_1 > \dots > k_m = 2\}$
Output: $\{k \mapsto \hat{\pi}_k \mid k \in K\}$

```

1 resultado  $\leftarrow \{\}$ 
2 while  $|V| > 1$  do
3    $\Omega \leftarrow \{v_0\}$  // semilla inicial
4   for  $j = 1$  to  $|V| - 2$  do
5      $u^* \leftarrow \arg \min_{u \in V \setminus \Omega} g(u, \Omega)$ 
6      $\Omega \leftarrow \Omega \cup \{u^*\}$ 
7   end
8    $u_{\text{ult}} \leftarrow V \setminus \Omega$ 
9   if  $|V| \in K$  then
10    resultado[ $|V|$ ]  $\leftarrow$  partición actual
11  end
12  Contraer  $(\Omega, u_{\text{ult}})$  en un nuevo vértice;  $V \leftarrow V \setminus \{u_{\text{ult}}\}$ 
13 end
14 return resultado

```

Proposición 7.3 (Captura de k -Particiones). *La k -partición $\hat{\pi}_k$ se captura cuando $|V| = k$ durante el Algoritmo 1. Los valores $k_1 > k_2 > \dots > k_m$ se capturan en ese orden. No son búsquedas independientes: k_1 se obtiene antes que k_m .*

Nota 7.3 (Optimalidad para $k > 2$). El Teorema 7.1 garantiza la bi-partición óptima ($k = 2$). Para $k > 2$, las k -particiones capturadas como estados intermedios **no** tienen garantía de optimalidad.

7.6. Complejidad

Proposición 7.4 (Complejidad de KQNodes). *El Algoritmo 1 realiza $n - 1$ fases con $\mathcal{O}(n)$ evaluaciones de (6) de costo $\mathcal{O}(2^n)$ cada una. Con memoización:*

$$T_{\text{KQNodes}}(n) = \mathcal{O}(n^2 \cdot 2^n) \quad (\text{práctico: } \mathcal{O}(n \cdot 2^n) \text{ con alta reutilización})$$

7.7. Ubicación en el Código

Elemento	Descripción
<code>class KQNodes(SIA)</code>	<code>QNodes/src/strategies/q_nodes.py</code>
<code>algorithm(vertices, ks)</code>	Punto de entrada; implementa el Algoritmo 1
<code>funcion_submodular(d, w)</code>	Evalúa $g(\Delta, \Omega)$ según (6)
<code>memoria_delta</code>	Caché dict de $(emd_{\Delta}, p_{\Delta})$
<code>memoria_particiones</code>	Caché de particiones ya evaluadas
<code>Worker subprocess</code>	<code>_worker_qnodes.py</code>

8. Algoritmo KGeoMIP

Idea general. KGeoMIP organiza todos los posibles estados del sistema como vértices de un “hipercubo”, donde cada arista conecta dos estados que difieren en exactamente un bit. Al recorrer este hipercubo nivel por nivel, el algoritmo calcula de forma eficiente la tabla de costos entre estados y luego usa esa tabla para aislar nodos uno a uno hasta obtener k grupos.

8.1. El Hipercubo de Hamming

En palabras sencillas. Con n nodos binarios hay 2^n estados posibles. El hipercubo de Hamming conecta esos estados según cuántos bits difieren: dos estados son vecinos (distancia 1) si solo difieren en un bit, por ejemplo 0010 y 0011. Los estados que difieren en exactamente ℓ bits forman el “nivel ℓ ” desde un estado base v_0 .

Definición 8.1 (Hipercubo de Hamming). El hipercubo de Hamming $\mathcal{H}_n = (\{0, 1\}^n, E)$ es un grafo donde dos estados son vecinos si y solo si difieren en exactamente un bit: $(x, y) \in E \Leftrightarrow d_H(x, y) = 1$. El **nivel de Hamming** ℓ desde un estado base v_0 agrupa todos los estados a distancia exactamente ℓ :

$$L_\ell(v_0) = \{w \in \{0, 1\}^n : d_H(w, v_0) = \ell\}, \quad |L_\ell(v_0)| = \binom{n}{\ell}$$

KGeoMIP recorre $\mathcal{H}_n$ desde v_0 nivel por nivel (primero los vecinos a distancia 1, luego a distancia 2, etc.), calculando el costo de cada transición una única vez y guardándolo en `tabla_transiciones`.

8.2. Tabla de Costos de Transición

En palabras sencillas. Para calcular el EMD entre dos estados i y j que están a distancia d en el hipercubo, KGeoMIP usa una fórmula recursiva: el costo entre i y j se construye promediando el costo entre j y sus vecinos intermedios hacia i . Esta recursión permite calcular toda la tabla de una sola pasada por el hipercubo, sin resolver un programa lineal separado para cada par.

Definición 8.2 (Costo de Transición Recursivo). Para dos estados i, j con $d_H(i, j) = d$, el vector de costos $\text{tx}(i, j) \in \mathbb{R}^{n_v}$ (una entrada por nodo del alcance) se calcula como:

$$\text{tx}(i, j) = \frac{1}{2^d} \left(|X[i] - X[j]| + \sum_{k \in \mathcal{N}(j, i)} \text{tx}(k, j) \right) \quad (7)$$

donde $X[v]$ es la fila v de la TPM (las probabilidades de cada nodo dado el estado v), $|\cdot|$ es la diferencia término a término, y $\mathcal{N}(j, i)$ son los estados vecinos de j a un paso más cerca de i . Caso base: dos estados iguales tienen costo cero, $\text{tx}(i, i) = \mathbf{0}$.

Proposición 8.1 (Costo de la Tabla). La tabla completa tiene $2^n - 1$ entradas, con costo $\mathcal{O}(n_v)$ por entrada:

$$T_{\text{tabla}}(n) = \mathcal{O}(2^n \cdot n_v) \approx \mathcal{O}(2^n \cdot n)$$

Una vez construida, cada consulta es $\mathcal{O}(1)$. Esta construcción domina el tiempo total de KGeoMIP para todo $n \gtrsim 10$.

8.3. Bisección Greedy Recursiva

Con la tabla lista, KGeoMIP obtiene la k -partición en $k - 1$ pasos: en cada paso aísla el nodo cuyo aislamiento produce el menor EMD. Empieza con todos los nodos en un grupo y los va separando uno a uno hasta llegar a k grupos.

Para cada candidato u , calcula cuánto cambia el EMD al sacarlo de su grupo actual y ponerlo solo:

$$\Delta\varphi(u, \hat{\pi}) = \text{EMD}[p, p_{\hat{\pi}'}] - \text{EMD}[p, p_{\hat{\pi}}] \quad (8)$$

donde $\hat{\pi}'$ es la partición que resulta de mover u a su propio grupo. Gracias a la tabla precalculada, esta actualización solo toca los “cubos” afectados por el movimiento de u , con costo $\mathcal{O}(n_v)$ en vez de recalcular todo desde cero ($\mathcal{O}(2^n)$).

Algoritmo 2: Bisección Greedy Recursiva de KGeoMIP

Input: Tabla tx;
vértices $V_v = V_{\text{pres}} \cup V_{\text{efec}}$;
entero $k \geq 2$
Output: k -partición $\hat{\pi}_k$

```

1  $\hat{\pi} \leftarrow \{V_v\}$  // un único grupo inicial
2 for  $r = 1$  to  $k - 1$  do
3    $u^* \leftarrow \arg \min_{u \in V_v} \text{EMD}[p, p_{\hat{\pi} \cup \{u\}}]$ 
4   Sea  $S_u$  el grupo actual de  $u$ 
5    $\hat{\pi} \leftarrow (\hat{\pi} \setminus \{S_u\}) \cup \{S_u \setminus \{u\}, \{u\}\}$ 
6 end
7 return  $\hat{\pi}$ 

```

Proposición 8.2 (Complejidad Total de KGeoMIP). *El paso greedy (Algoritmo 2) tiene costo $\mathcal{O}((k - 1) \cdot n_v^2)$, negligible frente a la tabla. La complejidad total es:*

$$T_{\text{KGeoMIP}}(n, k) = \mathcal{O}(2^n \cdot n) + \mathcal{O}(k \cdot n^2) = \mathcal{O}(2^n \cdot n)$$

Proposición 8.3 (Limitación de Optimalidad). *El Algoritmo 2 **no** garantiza $\hat{\pi}_k = \pi_k^*$. Cada elección u_r^* minimiza el EMD local en el paso r , pero puede existir una secuencia alternativa con una elección subglobal en el paso r que produzca φ_k estrictamente menor en los pasos siguientes. El ratio de aproximación $\varphi(\hat{\pi}_k)/\varphi_k^*$ no está acotado teóricamente.*

Ejemplo 8.1 (Contraejemplo al greedy). $n_v = 4$, vértices $\{a, b, c, d\}$, $k = 3$. Suponga $\text{EMD}[p, p_{\hat{\pi} \cup \{a\}}] = 0,10$ y $\text{EMD}[p, p_{\hat{\pi} \cup \{b\}}] = 0,15$. El greedy aísla a . Pero si el óptimo requiere aislar b primero para obtener $\{a, c\} \mid \{b\} \mid \{d\}$ con $\varphi = 0,05$, mientras que aislar a fuerza $\{a\} \mid \{b, c\} \mid \{d\}$ con $\varphi = 0,12$, el greedy retorna 0,12 cuando el óptimo es 0,05.

8.4. Ubicación en el Código

Elemento	Descripción
<code>class KGeoMIP(SIA)</code>	<code>geometric.py</code> — clase principal
<code>aplicar_estrategia(...)</code>	Punto de entrada; filtra ks válidos
<code>find_mip(ks)</code>	Orquesta tabla y bisección greedy
<code>calcular_costos_nivel()</code>	Recorre $\mathcal{H}_n$ por distancia Hamming
<code>calcular_costo()</code>	Calcula $tx(i, j)$ según (7)
<code>eval_isolate()</code>	$\Delta\varphi(u, \hat{\pi})$ según (8)
<code>tabla_transiciones</code>	dict de costos EMD por transición
<code>Worker subprocess</code>	<code>_worker_geomip_recurativo.py</code>

Nota 8.1 (Node-partitions en KGeoMIP). KGeoMIP opera sobre **node-partitions**: cada nodo $u \in V_v$ se asigna a *una sola* parte tanto para t como para $t+1$. El tamaño de este espacio es $2^{n_v-1} - 1$, notablemente menor que el espacio de cross-partitions de KQNodes ($2^{u+v-1} - 1$). Esta restricción permite usar la tabla de costos Hamming, pero implica que algunas particiones válidas según IIT 3.0 *no* son exploradas por KGeoMIP.

Nota 8.2 (Métrica real de KGeoMIP y GeneticKGeoMIP). Tanto KGeoMIP como GeneticKGeoMIP calculan el EMD con métrica de Hamming utilizando `pyemd` (programación lineal). El rango de φ depende de las distribuciones concretas del sistema y **no** está acotado por n_v . Los valores son incomparables con los de KQNodes (véase Nota 2.1).

9. Algoritmo GeneticKGeoMIP

Idea general. GeneticKGeoMIP es una búsqueda evolutiva: mantiene una “población” de particiones candidatas, las evalúa, y en cada generación combina y muta las mejores para explorar nuevas opciones. No garantiza encontrar la óptima, pero suele superar a KGeoMIP para $k \geq 3$ porque explora el espacio de forma más amplia.

9.1. Motivación

El paso greedy de KGeoMIP puede quedar atrapado en un mínimo local: una elección localmente buena en el paso r puede bloquear mejores particiones en pasos siguientes (véase Proposición 8.3 y Ejemplo 8.1). GeneticKGeoMIP explora más opciones a costa de mayor tiempo.

A diferencia de KGeoMIP, **no usa la tabla de costos Hamming**. Cada partición candidata se evalúa directamente llamando a `k_partir()` + `distribucion_marginal()` + `emd_efecto()`, con costo $\mathcal{O}(2^n)$ por evaluación.

9.2. Espacio de Búsqueda y Codificación

En palabras sencillas. Cada “individuo” de la población es una asignación de grupo para cada nodo. Con n_v nodos y k grupos, se representa como un vector de n_v números, cada uno entre 0 y $k - 1$. Por ejemplo, con 4 nodos y $k = 2$: $g = (0, 1, 0, 1)$ significa que los nodos 0 y 2 van al grupo 0, y los nodos 1 y 3 al grupo 1.

Definición 9.1 (Espacio de Búsqueda). El espacio de todas las posibles k -particiones de n_v nodos tiene:

$$|\mathcal{S}(n_v, k)| = S(n_v, k)$$

Para $n = 10$ ($n_v \approx 20$), $k = 3$: hay $\approx 5,8 \times 10^8$ particiones posibles; imposible revisarlas todas.

Cada individuo se codifica como:

$$g = (g_1, \dots, g_{n_v}) \in \{0, \dots, k - 1\}^{n_v}$$

Para evitar contar la misma partición varias veces (solo con etiquetas distintas), se usa **forma canónica**: el primer grupo que aparece siempre recibe la etiqueta 0, el segundo la 1, etc.

Definición 9.2 (Validez y Reparación). Un individuo es **válido** si ningún grupo queda vacío. Si tras un cruce o mutación algún grupo queda vacío, la función `repair(g, k)` lo arregla moviendo un nodo del grupo más poblado al grupo vacío.

9.3. Función de Aptitud

La **aptitud** (fitness) indica qué tan buena es una partición. Como queremos minimizar φ , se define al revés: cuanto menor sea φ , mayor será $f(g)$.

Definición 9.3 (Fitness).

$$f(g) = \frac{1}{1 + \varphi(\pi_g, v_t)} \in (0, 1] \quad (9)$$

El algoritmo **maximiza** f , lo que es equivalente a minimizar φ . Calcular $f(g)$ para un individuo requiere construir las distribuciones marginales de cada grupo p_{S_i} , combinarlas en p_{π_g} (Ec. 2) y resolver el programa lineal del EMD (Ec. 3), con costo $\mathcal{O}(2^n)$ por individuo. Para evitar repetir cálculos, los resultados se guardan en `eval_cache` con la partición como clave.

9.4. Operadores Genéticos

Selección por Torneo. Se eligen 3 individuos al azar de la población y se selecciona el mejor:

$$g^* = \arg \max_{g \in \mathcal{T}} f(g), \quad \mathcal{T} \subseteq P_t, |\mathcal{T}| = 3$$

Esto favorece a los más aptos sin descartar por completo a los menos aptos.

Cruce Basado en Grupos. Dados dos padres $g^{(1)}$ y $g^{(2)}$, se elige un subconjunto aleatorio de etiquetas de grupo $\mathcal{G}$. El hijo hereda del padre 1 los nodos que en el padre 1 pertenecen a $\mathcal{G}$, y del padre 2 el resto:

$$h_i = \begin{cases} g_i^{(1)} & \text{si } g_i^{(1)} \in \mathcal{G} \\ g_i^{(2)} & \text{en otro caso} \end{cases} \rightarrow h \leftarrow \text{repair}(h, k)$$

Mutación. Con probabilidad $p_m = 0,2$, se elige un nodo al azar y se le asigna un grupo distinto al que tenía:

$$h_i = c \sim \text{Uniform}(\{0, \dots, k-1\} \setminus \{g_i\}) \rightarrow h \leftarrow \text{repair}(h, k)$$

Elitismo. El mejor individuo de cada generación pasa directamente a la siguiente, garantizando que la mejor solución nunca empeora:

$$f(g_{t+1}^*) \geq f(g_t^*) \quad \forall t$$

Parada Temprana. Si la mejor aptitud no mejora más de 10^{-4} durante e generaciones consecutivas, el algoritmo termina antes del máximo de generaciones.

9.5. Parámetros Adaptativos

Cuadro 2: Parámetros de GeneticKGeoMIP en función de n_v

n_v	Población μ	Generaciones T	Parada temprana e
≤ 40	50	150	20
$\in (40, 60]$	70	120	22
> 60	100	100	25

9.6. Complejidad y Comparación con KGeoMIP

Proposición 9.1 (Complejidad de GeneticKGeoMIP).

$$T_{\text{Genético}}(n, k) = \mathcal{O}(\mu \cdot T \cdot 2^n)$$

Con la caché *eval_cache* (clave: *tuple(g)*), los individuos repetidos se evalúan en $\mathcal{O}(1)$.

Proposición 9.2 (Comparación de velocidad). Para los parámetros adaptativos, $\mu \cdot T \geq 5000 > n$ para todo $n \leq 25$, por lo que:

$$T_{\text{KGeoMIP}} = \mathcal{O}(n \cdot 2^n) < T_{\text{Genético}} = \mathcal{O}(\mu \cdot T \cdot 2^n)$$

KGeoMIP es asintóticamente más rápido. Sin embargo, GeneticKGeoMIP puede encontrar φ menores para $k \geq 3$, ya que el cruce basado en grupos explora regiones distintas de $\mathcal{S}(n_v, k)$, superando los óptimos locales del greedy (Ejemplo 8.1).

Nota 9.1. No existe garantía teórica de que GeneticKGeoMIP encuentre π_k^* (*No Free Lunch Theorem*). La superioridad sobre KGeoMIP es empírica y depende de la instancia concreta.

Nota 9.2 (Restricción práctica de alcance para GeneticKGeoMIP). La función de aptitud de GeneticKGeoMIP invoca `k_partir()` + `distribucion_marginal()` con costo $\mathcal{O}(2^{n_{\text{alc}}})$ por evaluación, donde n_{alc} es el número de nodos en el *alcance* (no el tamaño del mecanismo completo). Para $n_{\text{alc}} \geq 15$, la evaluación requiere matrices de $2^{15} = 32\,768$ filas y el tiempo se vuelve prohibitivo. Resultados experimentales con los sistemas de prueba (semilla 73) confirmaron que:

- $n_{\text{alc}} \leq 14$: viable, tiempos 3–120 s por prueba.
- $n_{\text{alc}} \geq 15$: inviable dentro de un límite práctico de 300 s.

El factor limitante es el tamaño del alcance, no el del sistema.

9.7. Ubicación en el Código

Elemento	Descripción
<code>class GeneticKGeoMIP(SIA)</code>	<code>genetic_optimizer.py</code>
<code>optimize(...)</code>	Punto de entrada; acepta parámetros sobreescritos
<code>group_based_crossover(p1,p2)</code>	Cruce con herencia de $ \mathcal{G} $ grupos
<code>mutate(genes, k)</code>	Mutación con reparación
<code>tournament_select()</code>	Torneo de tamaño 3
<code>repair(genes, k)</code>	Garantiza todos los grupos no vacíos
<code>eval_cache</code>	<code>dict</code> clave <code>tuple(g) → f(g)</code>
<code>ThreadPoolExecutor(4)</code>	Paralelización de evaluaciones
<code>Worker subprocess</code>	<code>_worker_geomip.py</code>

10. Clase Base SIA y Núcleo Compartido

10.1. Clase SIA

Método / Atributo	Descripción
<code>sia_preparar_subsistema(condicion, alcance, mecanismo, tpm)</code>	Marginaliza la TPM a variables relevantes; prepara ncubos y distribuciones marginales
<code>sia_subsistema</code>	Objeto <code>System</code> con el subsistema preparado
<code>sia_dists_marginales</code>	Distribuciones marginales — referencia para el EMD

10.2. Clase Solution

Atributo	Tipo	Descripción
<code>estrategia</code>	<code>str</code>	Nombre del algoritmo y k
<code>perdida</code>	<code>float</code>	Valor φ encontrado (= EMD de la partición)
<code>distribucion_subsistema</code>	<code>ndarray</code>	Distribución original del subsistema
<code>distribucion_particion</code>	<code>ndarray</code>	Distribución del sistema particionado
<code>tiempo_total</code>	<code>float</code>	Segundos de ejecución para este k
<code>particion</code>	<code>str</code>	Descripción textual de la partición encontrada

10.3. Funciones Clave del Núcleo

Función	Módulo	Descripción
<code>emd_efecto(dist1, dist2)</code>	<code>funcs/base.py</code>	EMD (3) entre dos distribuciones
<code>seleccionar_subestado()</code>	<code>funcs/base.py</code>	Extrae el subestado condicionado del sistema
<code>fmt_k_parte_q(partes)</code>	<code>funcs/format.py</code>	Formatea la k -partición como texto

11. Módulo de Ejecución por Lotes (scripts/run_batch.py)

11.1. Descripción

Procesa automáticamente pruebas de docs/DatosPruebas2026_1.xlsx sin intervención de la GUI. Usa los mismos workers de gui/logica/ como subprocesos independientes.

11.2. Diferencias con la GUI

Aspecto	GUI	run_batch.py
TPM	Manual, CSV o aleatoria por semilla	Solo aleatoria por semilla
Alias geomip	GeneticKGeoMIP	GeneticKGeoMIP
Alias geomip-recursivo	No disponible directamente	KGeoMIP greedy
Guardado	Manual (botón GUARDAR)	Automático tras cada prueba

11.3. Uso

```
# KQNodes, hoja 10A, pruebas 1 a 25
python scripts/run_batch.py --sheet 10A --strategy qnodes --desde 1 --
hasta 25

# KGeoMIP recursivo, k=2 3 4, timeout 5 min
python scripts/run_batch.py --sheet 15B --strategy geomip-recursivo --
ks 2 3 4 --timeout 300

# Hoja 25A solo pruebas seguras (mecanismo <= 17 nodos)
python scripts/run_batch.py --sheet 25A --strategy geomip --safe-only
```

12. API de Referencia

12.1. Interfaz Común

Listing 6: Firma de aplicar_estrategia

```
1 def aplicar_estrategia(
2     self,
3     condicion: str,          # cadena binaria n bits
4     alcance: str,           # cadena binaria n bits
5     mecanismo: str,         # cadena binaria n bits
6     tpm: np.ndarray,        # shape (2**n, n), dtype float
7     ks: list[int],          # e.g. [2, 3, 4, 5]
8 ) -> dict[int, Solution]:  # {k: Solution}
```

12.2. Ejemplo de Uso Directo

Listing 7: Uso directo de KGeoMIP

```
1 import sys, numpy as np
2 from pathlib import Path
```



```

3
4 sys.path.insert(0, str(Path("GeoMIP/src/kgeomip")))
5 from src.controllers.manager import Manager
6 from src.controllers.strategies.geometric import KGeoMIP
7
8 n = 5
9 tpm = np.random.randint(2, size=(2*n, n)).astype(float)
10
11 gestor = Manager(estado_inicial="10000")
12 resultados = KGeoMIP(gestor).aplicar_estrategia(
13     condicion="11111", alcance="11111", mecanismo="11111",
14     tpm=tpm, ks=[2, 3],
15 )
16
17 for k, sol in resultados.items():
18     print(f"k={k}: phi={sol.perdida:.6f} t={sol.tiempo_total:.2f}s")
19     print(sol.particion)

```

12.3. Parámetros Adicionales de GeneticKGeoMIP

Listing 8: Llamada con parámetros sobreescritos

```

1 GeneticKGeoMIP.optimize(
2     gestor, condicion, alcance, mecanismo, tpm, ks,
3     population_size=50,          # None -> adaptativo
4     generations=150,            # None -> adaptativo
5     tournament_size=3,
6     mutation_rate=0.2,
7     early_stop_generations=20, # None -> adaptativo
8 )

```

13. Análisis de Complejidad

La Tabla 3 consolida los tiempos registrados en las pruebas con los cinco sistemas (semilla 73) y estimaciones para combinaciones no ejecutadas, basadas en la tasa de crecimiento observada. Los tiempos son **rangos** porque dependen del tamaño real del alcance (n_{alc}) y el mecanismo dentro de cada sistema: pruebas con alcances pequeños dentro de un sistema grande son rápidas, mientras que las que usan el sistema completo son las más lentas.

Nota 13.1 (Por qué k casi no afecta el tiempo). Para KQNodes y KGeoMIP, el cuello de botella es construir la tabla de distancias ($\mathcal{O}(2^{n_{\text{alc}}} \cdot n_{\text{alc}})$), no la búsqueda de la partición. Por eso el tiempo para $k=2$ y $k=5$ es prácticamente idéntico. El Genético sí crece con k (más tipos de partición que evaluar): en las pruebas, pasar de $k=2$ a $k=5$ multiplicó el tiempo por un factor de 4–5.

Nota 13.2 (KGeoMIP es más rápido que KQNodes para el mismo alcance). KGeoMIP recorre $2^{n_{\text{alc}}-1} - 1$ node-partitions. KQNodes recorre $2^{n_u+n_v-1} - 1$ cross-partitions (espacio estrictamente mayor). Por eso KGeoMIP termina en 0.03–115 s en el sistema de 20 nodos mientras KQNodes tarda hasta 226 s sobre las mismas pruebas.

Cuadro 3: Tiempos con alcance completo ($n_{\text{alc}} = n$), rango $k=2,5$. * = estimado según el crecimiento observado entre sistemas.

Sistema	KQNodes	KGeoMIP	Genético
10A ($n=10$)	0.03–0.15 s	<0.01–0.06 s	0.3–2.5 s
15B ($n=15$)	0.10–2.6 s	0.01–1.4 s	0.6–10 s
20A ($n=20$)	0.3–226 s	0.05–115 s	~5–20 min*
22A ($n=22$)	0.5–311 s	0.08–330 s	~30 min–2 h*
25A ($n=25$)	~40 min*	~35 min	>2 h*

Nota 13.3 (Genético: más lento a mayor alcance dentro de un sistema grande). El Genético ejecuta miles de evaluaciones de partición, cada una calculando la distribución marginal sobre la TPM completa del sistema (tamaño 2^n). Por eso en el sistema de 20 nodos, una prueba con $n_{\text{alc}} = 14$ tarda hasta 36 s, mientras que para $n_{\text{alc}} = 15$ en el sistema de 15 nodos la misma TPM de solo 2^{15} estados permite terminar en menos de 10 s. El costo del Genético crece con n (sistema) y con n_{alc} (alcance) a la vez: duplicar cualquiera de los dos puede multiplicar el tiempo por 3–8 según la convergencia del algoritmo genético.

13.1. Guía de Selección de Algoritmo

Cuadro 4: Algoritmo recomendado según tamaño del sistema (n) y k deseado. KGeoMIP y Genético comparten la misma métrica EMD (resultados comparables); KQNodes usa una métrica distinta.

Tamaño	$k=2$	$k=3$	$k=4, 5$
$n \leq 10$	Cualquiera (<0.2 s)	Cualquiera (<2.5 s)	Cualquiera (<2.5 s)
$n = 11.,15$	KGeoMIP (más rápido) o KQNodes	KGeoMIP o KQNodes	Genético (<10 s)
$n = 16.,20, n_{alc} \leq 14$	KGeoMIP	KGeoMIP	Genético (<1 min)
$n = 16.,20, n_{alc} > 14$	KGeoMIP (<2 min)	KGeoMIP (<2 min)	KGeoMIP; Genético estim. 5–20 min
$n = 21.,22$	KGeoMIP (<6 min)	KGeoMIP (<6 min)	KGeoMIP (<6 min)
$n = 23.,25$	KGeoMIP (el más rápido medido)	KGeoMIP	KGeoMIP

14. Limitaciones y Trabajo Futuro

14.1. Limitaciones Conocidas

Limitación	Descripción	Impacto
Ningún algoritmo garantiza φ óptimo para $k > 2$	KQNodes: captura intermedias del proceso de fusión. KGeoMIP: greedy explícito. Genético: heurístico estocástico.	$\varphi(\hat{\pi}_k) \geq \varphi_k^*$ siempre
Métrica distinta en KQNodes	KQNodes usa L^1 sobre marginales por nodo (rango $[0, n_{\text{alc}}]$). KGeoMIP y Genético usan EMD Wasserstein-1 con Hamming. Los valores de φ no son comparables entre algoritmos.	Normalizar KQNodes por n_{alc} ; no comparar φ crudo
Espacio de particiones reducido en KGeoMIP/Genético	Ambos limitan a node-partitions; KQNodes explora cross-partitions. KQNodes es más fiel a IIT 3.0 en el espacio de búsqueda.	Particiones encontradas por KGeoMIP pueden no existir en KQNodes
Límite de RAM para $n \geq 23$	La TPM de $n = 23$ tiene $2^{23} \times 23 \approx 193$ M entradas ($\sim 1,5$ GB).	MemoryError en máquinas con < 8 GB libres
GUI mono-ejecución	Un Runner , un hilo, un análisis a la vez.	Para lotes grandes usar run_batch.py
k máximo = 5	Limitación de GUI y run_batch.py ; los algoritmos soportan k hasta n_v .	Modificar código directamente para $k > 5$

14.2. Trabajo Futuro

- Implementar búsqueda con backtracking para KGeoMIP $k > 2$ (reemplazar greedy puro y acotar el ratio de aproximación).
- Paralelizar la evaluación de candidatos en KGeoMIP.
- Exportar resultados en JSON para integración con herramientas de análisis IIT.
- Visualización de k -particiones en el hipercubo.
- **Unificar métricas:** reemplazar **emd_efecto** en KQNodes por EMD real con Hamming (vía **pyemd**) para permitir comparación directa con KGeoMIP/Genético. Requiere calcular la distribución conjunta marginalizada en lugar de las marginales por nodo.
- Extender KGeoMIP para explorar cross-partitions (espacio más amplio, fiel a IIT 3.0) manteniendo la tabla de costos Hamming.